

Constraint Satisfaction II

Lecture 9

CS 4880

Kirk Duran

Today

- ▶ Why correct backtracking can still explore an enormous search tree.
- ▶ How MRV, the degree heuristic, and LCV improve variable and value ordering.
- ▶ How forward checking detects immediate consequences of an assignment.
- ▶ How arc consistency and AC-3 propagate constraints across neighboring variables.
- ▶ How Maintaining Arc Consistency integrates propagation into recursive search.
- ▶ How backjumping and min-conflicts provide alternative ways to recover from or avoid bad decisions.

Key idea

Efficient CSP solving is not a different problem formulation; it is the systematic use of ordering and inference to avoid searching assignments that constraints have already ruled out.

Part 1: Why Naive Backtracking Is Not Enough

The Baseline Solver We Already Have

Lecture 8 gave us a complete recursive solver:

1. choose an unassigned variable;
2. try a value from its domain;
3. reject values inconsistent with the current assignment;
4. recurse;
5. undo the choice after failure.

Correctness is not the issue

With finite domains, basic backtracking is complete. The problem is how much of the search tree it may explore before finding a solution or proving failure.

Image placeholder

Backtracking search tree with a small successful branch hidden among many explored branches.

Where Does Backtracking Waste Time?

Variable choice

We may assign an easy variable while a nearly impossible variable is waiting elsewhere.

Value choice

We may choose a legal value that destroys most of the remaining flexibility.

Late failure

We may continue searching long after the current assignments logically imply a contradiction.

Key idea

The next improvements target three questions: where should we search, which value should we try, and what can we infer before searching further?

Three Levers for a Better CSP Solver

Lever	Question	Techniques
Variable ordering	Which variable next?	MRV, degree heuristic
Value ordering	Which value first?	LCV
Inference	What follows now?	Forward checking, AC-3, MAC

Key idea

These techniques preserve the CSP itself; they change the order and amount of work needed to solve it.

Part 2: Variable and Value Ordering — Search Where Failure Is Informative



Minimum Remaining Values (MRV)

Definition

The **Minimum Remaining Values (MRV)** heuristic chooses the unassigned variable with the smallest number of currently legal values.

$$X^* = \arg \min_{X \text{ unassigned}} |D_X|.$$

- ▶ A small domain means the variable is highly constrained by the current partial assignment.
- ▶ If a contradiction is inevitable, MRV tries to expose it early.
- ▶ MRV is therefore often called the *fail-first* heuristic.

Key idea

Search the most fragile decision first rather than postponing it until after many irrelevant assignments.

Worked MRV Example

Suppose a partial map-coloring assignment leaves:

$$D_{SA} = \{B\},$$

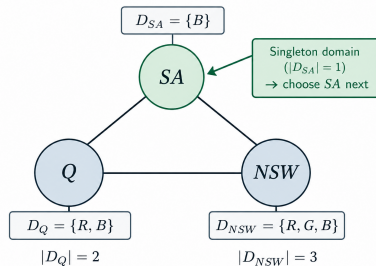
$$D_Q = \{R, B\},$$

$$D_{NSW} = \{R, G, B\}.$$

MRV chooses SA because

$$|D_{SA}| = 1.$$

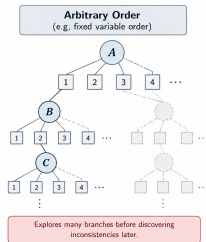
Assigning SA now either succeeds immediately or exposes a contradiction immediately.



Why MRV Can Shrink the Search Tree

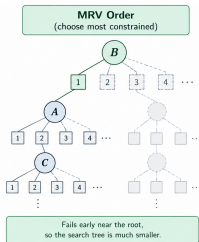
Arbitrary order

Choose a variable with four legal values, branch four ways, and discover much later that another variable had no valid continuation.



MRV order

Choose the nearly forced variable first. A dead end may be discovered before those four branches are ever created.



Degree Heuristic

Definition

The **degree heuristic** chooses the unassigned variable involved in the largest number of constraints on other unassigned variables.

- ▶ It is commonly used as a tie-breaker when several variables have the same MRV score.
- ▶ A high-degree variable can influence many remaining decisions.
- ▶ Assigning it early can create useful pruning elsewhere in the graph.

Australia intuition

South Australia is highly connected, whereas Tasmania is isolated. If both have equally large domains, assigning South Australia is usually more informative.

MRV and Degree Are Different Questions

Heuristic	Looks at	Intuition
MRV	current domain size	choose the most constrained variable
Degree	remaining graph connectivity	choose the most constraining variable

- ▶ MRV is dynamic: domains may shrink as assignments and inference accumulate.
- ▶ Degree is also dynamic if we count only constraints involving still-unassigned variables.
- ▶ A common policy is MRV first, then degree as the tie-breaker.

Key idea

MRV asks where failure is most likely; degree asks where one decision can affect the largest portion of the remaining problem.

Least Constraining Value (LCV)

Definition

The **Least Constraining Value (LCV)** heuristic orders the values of a chosen variable by how few values they eliminate from neighboring unassigned variables.

- ▶ Variable ordering tries to expose difficult variables early.
- ▶ Value ordering tries to preserve flexibility after that variable is chosen.
- ▶ LCV prefers values that leave neighbors with many alternatives.

Key idea

Choose the hardest variable first, then try the value that makes life easiest for the remaining variables.

Worked LCV Example

Suppose Q has domain

$$D_Q = \{R, G, B\}.$$

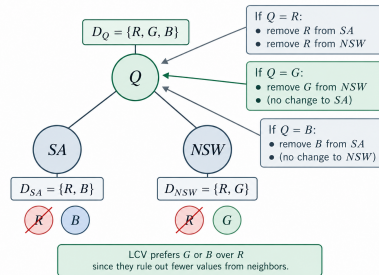
Its unassigned neighbors have domains:

$$D_{SA} = \{R, B\}, \quad D_{NSW} = \{R, G\}.$$

Under inequality constraints:

- ▶ $Q = R$ removes one value from both neighbors;
- ▶ $Q = G$ removes no value from SA and one from NSW ;
- ▶ $Q = B$ removes one from SA and none from NSW .

LCV prefers G or B before R .



Ordering Heuristics Change the Search Tree

Typical policy

```
X ← unassigned variable chosen by MRV  
break MRV ties with the degree heuristic  
for value in DOMAIN(X) ordered by LCV:  
  try X=value
```

- ▶ None of these heuristics removes a legal solution from the search space.
- ▶ They only alter the order in which equivalent search alternatives are explored.
- ▶ Their effectiveness comes from reaching contradictions or promising assignments sooner.

Key idea

Ordering helps us search more intelligently; inference goes further by deleting values that no longer need to be searched at all.

Part 3: Forward Checking — Infer Immediate Consequences

Forward Checking

Definition

Forward checking propagates each new assignment to its unassigned neighbors by deleting neighbor values inconsistent with that assignment.

After assigning $X = v$:

1. inspect each unassigned neighbor Y of X ;
2. remove every $y \in D_Y$ for which the constraint $C_{XY}(v, y)$ is false;
3. fail immediately if any neighbor domain becomes empty.

Key idea

A constraint violation need not wait until the neighbor is assigned; forward checking can predict the violation from the domain itself.

Forward Checking on Australia

Initially:

$$D_{WA} = D_{NT} = D_{SA} = \{R, G, B\}.$$

Assign

$$WA = R.$$

Because

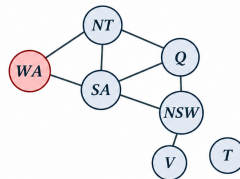
$$WA \neq NT, \quad WA \neq SA,$$

forward checking updates:

$$D_{NT} \leftarrow \{G, B\},$$

$$D_{SA} \leftarrow \{G, B\}.$$

After forward checking



Variable	Domain D_X
WA	$\{R\}$
NT	$\{\textcolor{red}{R}, G, B\}$
SA	$\{\textcolor{red}{R}, G, B\}$
Q	$\{R, G, B\}$
NSW	$\{R, G, B\}$
V	$\{R, G, B\}$
T	$\{R, G, B\}$

Assign $WA = R$

Remove R from unassigned neighbors (NT and SA)

Failure Can Be Detected Before the Next Assignment

Suppose a neighbor currently has

$$D_Y = \{R\}$$

and a newly assigned adjacent variable becomes

$$X = R$$

under constraint $X \neq Y$.

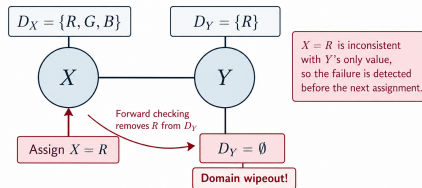
Forward checking removes R from D_Y :

$$D_Y \leftarrow \emptyset.$$

The current branch is impossible even though Y has not yet been assigned.

Key idea

An empty domain is a proof that the current partial assignment cannot be extended to a solution.



Forward Checking Inside Backtracking

Algorithm sketch

```
for value in ORDERED-DOMAIN-VALUES(X):  
    if value is consistent with assignment:  
        assign X=value  
        pruned ← FORWARD-CHECK(X, value)  
        if no domain is empty:  
            result ← BACKTRACK(assignment)  
            if result ≠ failure: return result  
        RESTORE(pruned)  
        remove X=value
```

Key idea

Inference changes the mutable domains, so every recursive branch must be able to undo exactly the values that branch pruned.

Implementation Detail: Domain Restoration

Unsafe approach

Permanently delete inconsistent values from shared domain sets.

After backtracking, later branches inherit pruning that was justified only by the abandoned assignment.

Safe approaches

- ▶ Record each removed $(variable, value)$ pair and restore it.
- ▶ Copy domains for each recursive call.
- ▶ Use a reversible trail / stack of domain changes.

Key idea

Correct CSP implementations distinguish global problem constraints from temporary domain reductions caused by the current search branch.

Part 4: Arc Consistency and Constraint Propagation

Forward Checking Stops After One Step

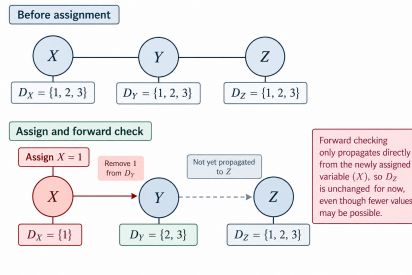
Forward checking only propagates directly from the variable that was just assigned.

Suppose:

$$X \rightarrow Y \rightarrow Z.$$

Assigning X may shrink D_Y .

That smaller D_Y may make some values in D_Z impossible, even though Z is not adjacent to the newly assigned variable X .



Key idea

Constraint propagation repeatedly applies local reasoning until no additional domain values can be removed.

Arc Consistency — Formal Definition

Definition

A directed arc $X_i \rightarrow X_j$ is **arc-consistent** if every value $x \in D_i$ has at least one value $y \in D_j$ such that the binary constraint $C_{ij}(x, y)$ is satisfied.

$$\forall x \in D_i, \exists y \in D_j : C_{ij}(x, y) = \text{true}.$$

- ▶ The condition is directional: $X_i \rightarrow X_j$ and $X_j \rightarrow X_i$ are separate arcs.
- ▶ A value with no support in the neighboring domain can be deleted.
- ▶ Domain deletions may invalidate support on other arcs.

Support Example

Let

$$D_X = \{1, 2, 3\}, \quad D_Y = \{2, 3\}$$

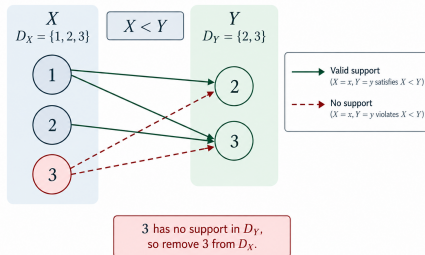
with constraint

$$X < Y.$$

For $X \rightarrow Y$:

- ▶ 1 is supported by 2 or 3;
- ▶ 2 is supported by 3;
- ▶ 3 has no support.

Therefore remove 3 from D_X .



The REVISE Operation

REVISE(X_i, X_j)

```
revised  $\leftarrow$  false
for each  $x$  in  $D_i$ :
    if no  $y$  in  $D_j$  satisfies  $C_{ij}(x, y)$ :
        remove  $x$  from  $D_i$ 
        revised  $\leftarrow$  true
return revised
```

- ▶ REVISE changes only D_i .
- ▶ It answers whether D_i changed, because a change may affect other neighboring arcs.
- ▶ If D_i becomes empty, the current CSP state is inconsistent.

AC-3: Propagate Until Stable

Definition

AC-3 repeatedly revises directed arcs until every arc is consistent or some domain becomes empty.

High-level algorithm

```
queue  $\leftarrow$  all directed arcs in the CSP
while queue is not empty:
     $(X_i, X_j) \leftarrow \text{queue.pop}()$ 
    if REVISE( $X_i, X_j$ ):
        if  $D_i$  is empty: return failure
        for each  $X_k$  in NEIGHBORS( $X_i$ ) except  $X_j$ :
            queue.add( $(X_k, X_i)$ )
return success
```

Why Re-Add Neighboring Arcs?

Suppose revising $Y \rightarrow Z$ removes a value from D_Y .
A value in D_X that previously depended on that deleted Y value may now have no support.

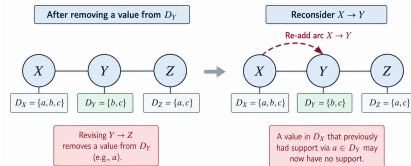
Therefore the arc

$$X \rightarrow Y$$

must be reconsidered.

Fixed-point view

AC-3 continues until every remaining value has local support and no revision can delete anything else.



Fixed-point view

AC-3 continues until every remaining value has local support and no revision can delete anything else.

AC-3 Trace: Initial Domains

Consider three variables:

$$D_X = \{1, 2, 3\}, \quad D_Y = \{1, 2, 3\}, \quad D_Z = \{2\}$$

with constraints

$$X < Y, \quad Y < Z.$$

Start with arcs:

$$X \rightarrow Y, Y \rightarrow X, Y \rightarrow Z, Z \rightarrow Y.$$

Revising $Y \rightarrow Z$ leaves only

$$D_Y = \{1\}.$$

Image placeholder

Three-node chain $X \rightarrow Y \rightarrow Z$ with domains shown;
highlight $Z=2$ and the revision $Y \rightarrow Z$ pruning Y
to 1.

AC-3 Trace: Propagation Reveals Failure

With

$$D_Y = \{1\},$$

reconsider the neighboring arc

$$X \rightarrow Y.$$

The constraint is $X < Y$, but no value in

$$D_X = \{1, 2, 3\}$$

is less than 1.

Therefore

$$D_X \leftarrow \emptyset.$$

Key idea

AC-3 derives a contradiction without assigning X or Y in search; the existing domains and constraints are already sufficient to prove failure.

AC-3 Cost and Benefit

Cost

For a binary CSP with at most d values per domain and e directed arcs, a straightforward AC-3 implementation has worst-case time on the order of

$$O(ed^3).$$

Benefit

- ▶ deletes values before branching;
- ▶ can expose contradictions early;
- ▶ may dramatically reduce the later backtracking tree.

Key idea

Constraint propagation deliberately spends computation now to avoid potentially exponential search later.

Forward Checking vs. Arc Consistency

	Forward Checking	Arc Consistency
Trigger	new assignment	domain changes / queued arcs
Propagation	assigned variable to neighbors	repeatedly across neighboring arcs
Cost	lower	higher
Pruning strength	weaker	stronger
Indirect contradictions	may miss	may expose

Key idea

Forward checking looks one step ahead; arc consistency repeatedly propagates the consequences of shrinking domains.

Maintaining Arc Consistency (MAC)

Definition

Maintaining Arc Consistency (MAC) runs arc-consistency propagation after each tentative assignment during backtracking search.

1. Choose a variable and tentative value.
2. Restrict that variable's domain to the chosen value.
3. Run AC-style propagation on affected arcs.
4. Recurse only if every domain remains nonempty.
5. Restore pruned values when backtracking.

Key idea

MAC tightly couples search and inference: every decision is immediately followed by the strongest local consequences the solver knows how to derive.

Backtracking With Heuristics and MAC

Integrated solver

```
BACKTRACK(assignment, domains):  
    if assignment is complete: return assignment  
    X ← SELECT-VARIABLE(MRV, degree)  
    for value in ORDER-VALUES(X, LCV):  
        if value is consistent:  
            assign X=value  
            restrict  $D_X$  to {value}  
            if MAC(domains) succeeds:  
                result ← BACKTRACK(...)  
                if result ≠ failure: return result  
            restore domains and assignment  
    return failure
```

Key idea

A practical CSP solver is a composition of search order, reversible domain state, and inference.

Part 5: Recovering More Intelligently From Failure



Chronological Backtracking Can Undo the Wrong Decision

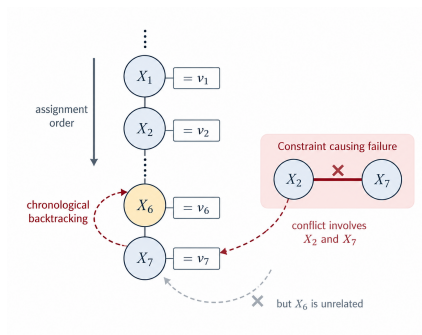
Standard backtracking returns to the most recently assigned variable.

But suppose a contradiction is caused by assignments to

$$X_2 \quad \text{and} \quad X_7,$$

while X_6 has no relationship to the failed constraint.

Chronological backtracking may first reconsider X_6 even though changing it cannot repair the conflict.



Backjumping

Definition

Backjumping uses information about which earlier assignments contributed to a failure and jumps back to a relevant conflicting variable rather than always returning one level.

- ▶ The solver records or reconstructs a conflict set associated with failure.
- ▶ Variables unrelated to the conflict can be skipped.
- ▶ More advanced variants combine conflict information across failures.

Key idea

Ordinary backtracking asks, “what did I choose most recently?” Backjumping asks, “which choice actually caused this contradiction?”

Part 6: Min-Conflicts — Local Search for Constraint Satisfaction

A Different CSP Strategy

Backtracking family

- ▶ starts with a partial assignment;
- ▶ maintains consistency while adding values;
- ▶ can systematically prove failure.

Local-search family

- ▶ starts with a complete assignment;
- ▶ allows temporary constraint violations;
- ▶ repeatedly repairs conflicts.

Key idea

CSPs can be solved either by constructing a legal assignment or by repairing an illegal complete assignment.

Min-Conflicts

Definition

Min-conflicts is a local-search method that repeatedly selects a conflicted variable and assigns the value that minimizes the number of violated constraints.

Algorithm

```
current ← a complete assignment
repeat up to max_steps:
  if current satisfies every constraint: return current
  X ← randomly choose a conflicted variable
  value ← value minimizing CONFLICTS(X, value, current)
  set X=value
return failure
```

Min-Conflicts on N-Queens

Use one variable per column:

$$Q_1, Q_2, \dots, Q_n.$$

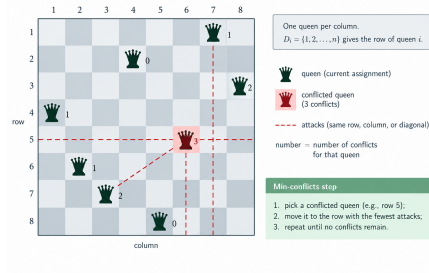
Domain:

$$D_i = \{1, 2, \dots, n\}$$

gives the row position of queen i .

At each step:

1. choose a queen currently attacking another queen;
2. move it to the row that creates the fewest attacks;
3. repeat until no conflicts remain.



When Min-Conflicts Is Attractive

Strengths

- ▶ very little memory;
- ▶ simple local updates;
- ▶ often effective when solutions are plentiful;
- ▶ natural for large assignment problems with easy repair moves.

Limitations

- ▶ incomplete under a finite step limit;
- ▶ may cycle or become trapped;
- ▶ does not naturally prove unsatisfiability;
- ▶ performance depends on landscape structure and restart strategy.

Key idea

Min-conflicts trades systematic guarantees for the ability to move through very large complete-state spaces cheaply.

Sudoku: Systematic Search or Min-Conflicts?

Backtracking + propagation

- ▶ exploits forced values aggressively;
- ▶ naturally handles tightly constrained puzzles;
- ▶ can establish failure when no solution exists.

Min-conflicts

- ▶ begins from a complete grid assignment;
- ▶ repairs row, column, and box conflicts;
- ▶ may struggle when legal configurations occupy narrow regions of the state space.

Key idea

The best CSP algorithm depends on whether the problem rewards logical propagation, local repair, or some combination of both.

Part 7: Applications Beyond Static Puzzles

Scheduling as a CSP

A scheduling problem can assign each activity a time, room, or resource.

Variables

Courses, jobs, meetings, or tasks.

Domains

Feasible times, rooms, machines, or workers.

Constraints

No overlap, precedence, capacity, availability, and resource limits.

Image placeholder

Timetable or job-shop diagram with overlapping assignments marked invalid and precedence arrows between selected tasks.

Key idea

The same heuristics and propagation mechanisms used for maps and Sudoku also apply to real allocation and scheduling systems.

What If Constraints Are Revealed Over Time?

Classical CSPs assume the variables, domains, and constraints are known before solving.

Some interactive problems instead reveal information after each action:

- ▶ diagnosis through tests;
- ▶ preference elicitation;
- ▶ planning with feedback;
- ▶ word-guessing games.

The solver repeatedly updates domains and constraints as new observations arrive.

Image placeholder

Interactive loop: choose action/guess $\rightarrow$
receive feedback $\rightarrow$ update
domains/constraints $\rightarrow$ choose again.

Wordle as Constraint Accumulation

Treat the five letter positions as variables with letter domains.

Feedback introduces constraints:

- ▶ **green**: the letter is fixed at that position;
- ▶ **yellow**: the letter occurs elsewhere but not at that position;
- ▶ **gray**: the guessed occurrence is excluded, subject to repeated-letter counts.

Each guess filters the remaining candidate assignments.

Image placeholder

Wordle-style five-letter grid with green, yellow, and gray feedback, alongside shrinking candidate letter domains.

Key idea

Constraint reasoning remains useful even when the complete constraint set is discovered incrementally rather than given all at once.

Part 8: Putting the Solver Together

Architecture of a Practical Backtracking Solver

Image placeholder

Pipeline diagram: BACKTRACK -> SELECT VARIABLE (MRV + degree) -> ORDER VALUES (LCV) -> ASSIGN
-> INFERENCE (forward checking or MAC/AC-3) -> recurse / restore on failure.

What Each Technique Contributes

Technique	Main question answered
MRV	Which variable is closest to failure?
Degree heuristic	Which variable constrains the most remaining variables?
LCV	Which value preserves the most flexibility?
Forward checking	What immediate neighbor values became impossible?
AC-3	What additional impossibilities propagate through the graph?
MAC	How do we maintain propagation throughout search?
Backjumping	Which earlier decision actually caused this failure?
Min-conflicts	Can a complete assignment be repaired instead?

Choosing a CSP Strategy

Prefer systematic search when

- ▶ completeness matters;
- ▶ proving unsatisfiability matters;
- ▶ strong propagation is available;
- ▶ constraints make many values forced or impossible.

Prefer local repair when

- ▶ the state space is enormous;
- ▶ complete assignments are easy to construct;
- ▶ conflicts are cheap to evaluate and repair;
- ▶ finding a good solution matters more than proving failure.

Key idea

Constraint satisfaction is a modeling framework; the solving strategy should match the guarantees and structure the application actually requires.

Constraint Satisfaction I and II

Lecture 8: Foundation

Variables, domains, constraints
Partial and complete assignments
Constraint graphs
Consistency intuition
Basic backtracking
Map coloring and Sudoku modeling

Lecture 9: Efficient Solving

MRV, degree, LCV
Domain-aware ordering
Forward checking
Arc consistency and AC-3
MAC and reversible inference
Backjumping and min-conflicts

Key idea

The CSP representation becomes powerful when the solver uses constraint structure not only to test assignments, but to decide what to search and what can be inferred without search.

Takeaways

- ▶ MRV and the degree heuristic choose informative variables; LCV preserves flexibility when choosing values.
- ▶ Forward checking removes values made immediately inconsistent by a new assignment.
- ▶ Arc consistency requires every value to have support in each neighboring domain.
- ▶ AC-3 repeatedly applies REVISE until a fixed point or an empty domain is reached.
- ▶ MAC embeds arc-consistency propagation inside backtracking and requires reversible domain updates.
- ▶ Backjumping uses conflict information to avoid undoing irrelevant decisions.
- ▶ Min-conflicts solves CSPs from the opposite direction by repairing complete but inconsistent assignments.
- ▶ Effective CSP solving combines representation, search order, inference, and problem-specific structure.

Key idea

The central algorithmic lesson of CSPs is that reasoning can remove large regions of a combinatorial search space before those regions are ever explored.